ARCHITECTURE OF AI SYSTEMS · DAY 2

Agentic AI Architecture.

From a model in a pipeline to a model that acts.
Harnesses, patterns, evaluation, context.

An agent is a model plus a harness.

Harness engineering.
Design patterns.
Architecture evaluation.
Context engineering.
Always-on agents.

PRESENTERS

Ricardo · Tomás · Simon · Luca · Matheus

OPENING

Five questions in forty minutes

Each presenter answers one question, grounded in one paper.

TIME	PRESENTER	QUESTION	MAIN SOURCE
0–2	All	Why go one layer up from Day 1?	—
2–9	Ricardo	What is a harness?	Barbaste et al. (2026)
9–15	Tomás	Which patterns describe an agent?	Liu et al. (2025)
15–21	Simon	How do we evaluate its architecture?	Lu et al. (2026)
21–29	Luca	What does context engineering look like?	Mohsenimofidi et al. (2026)
29–37	Matheus	What changes when agents run always-on?	Pi Durable · Hermes
37–40	All	Discussion	—

KEY CONCEPTS

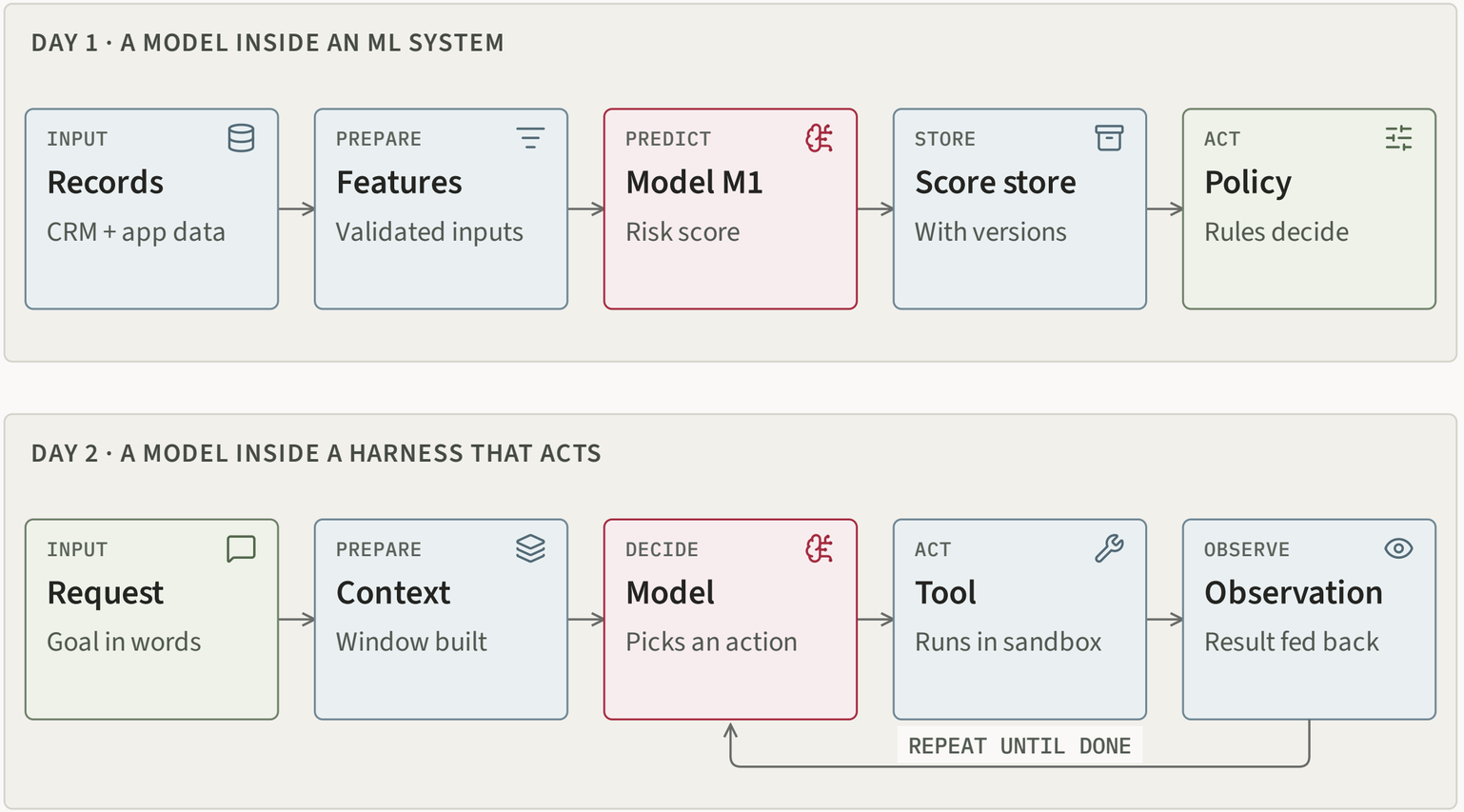
- **Five topics**, each self-contained: one question, one main source.
- **Real systems as evidence:** Pi, Pi Durable, Hermes and the eleven harnesses of one source-code study.
- **Topic-led:** no single running example this time.
- **Discussion** reserved for the last three minutes.

Same question as Day 1, one layer up: what must surround the model?

OPENING

One layer up from Day 1

Day 1 put a model inside a system. Day 2 puts it inside a loop that acts.



KEY CONCEPTS

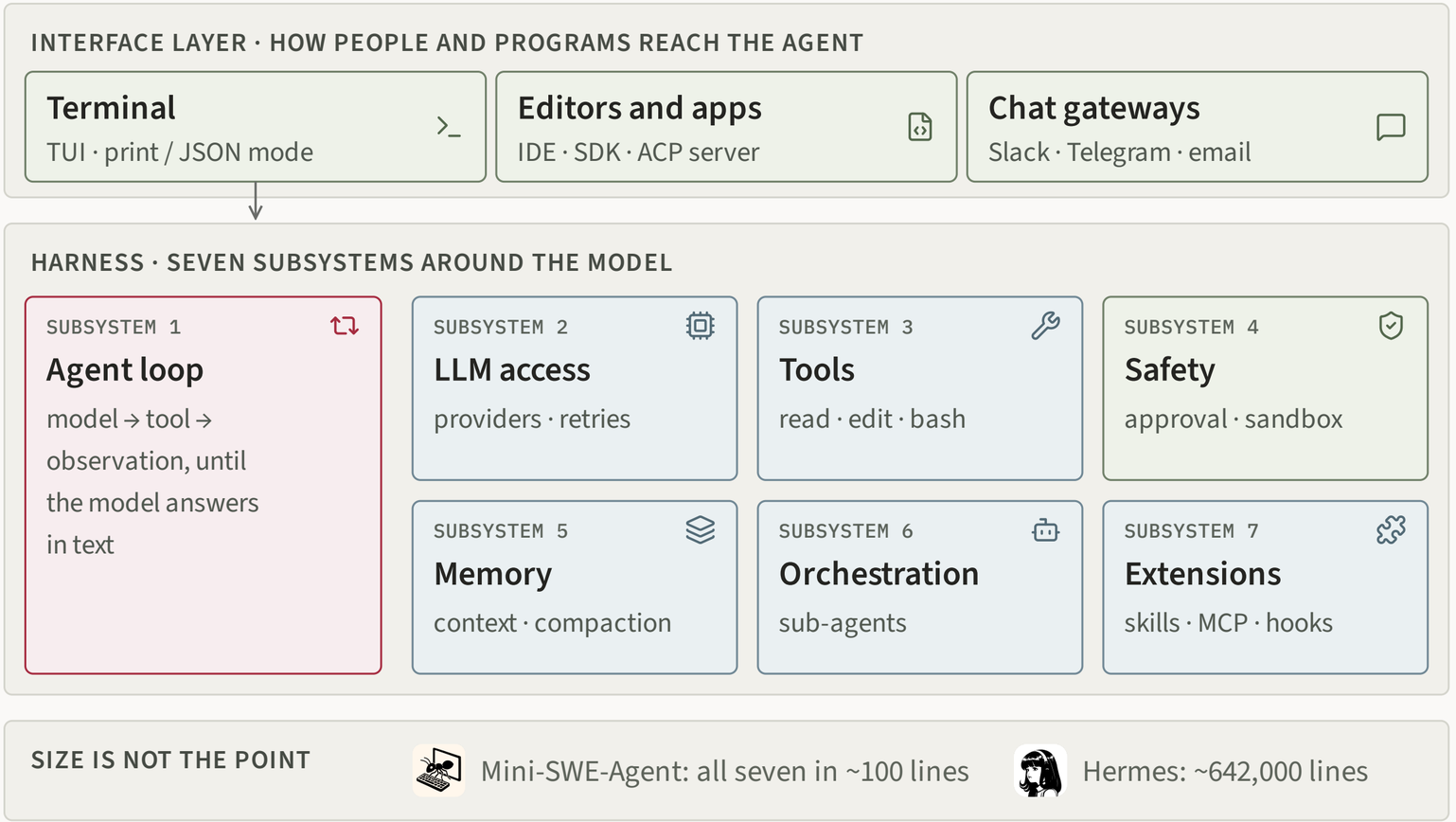
- **Day 1:** a one-way path from records to a business decision.
- **Day 2:** the model chooses an action, not just a score.
- **The return arrow** is new: results flow back as context.
- **Same engineering question:** what must surround the model?

An agent is still a model inside an architecture — now one that loops.

TOPIC 1 / HARNESS ENGINEERING

Agent = model + harness

The harness is everything except the model. A source-code study of eleven harnesses found the same seven subsystems in all of them.



KEY CONCEPTS

- **Agent loop** at the centre: model → tool → observation.
- **Six more subsystems:** LLM access, tools, safety, memory, orchestration, extensions.
- **Every harness takes a position** on all seven, even if it is “none”.
- **Interfaces** sit on top: terminal, editors, chat gateways.

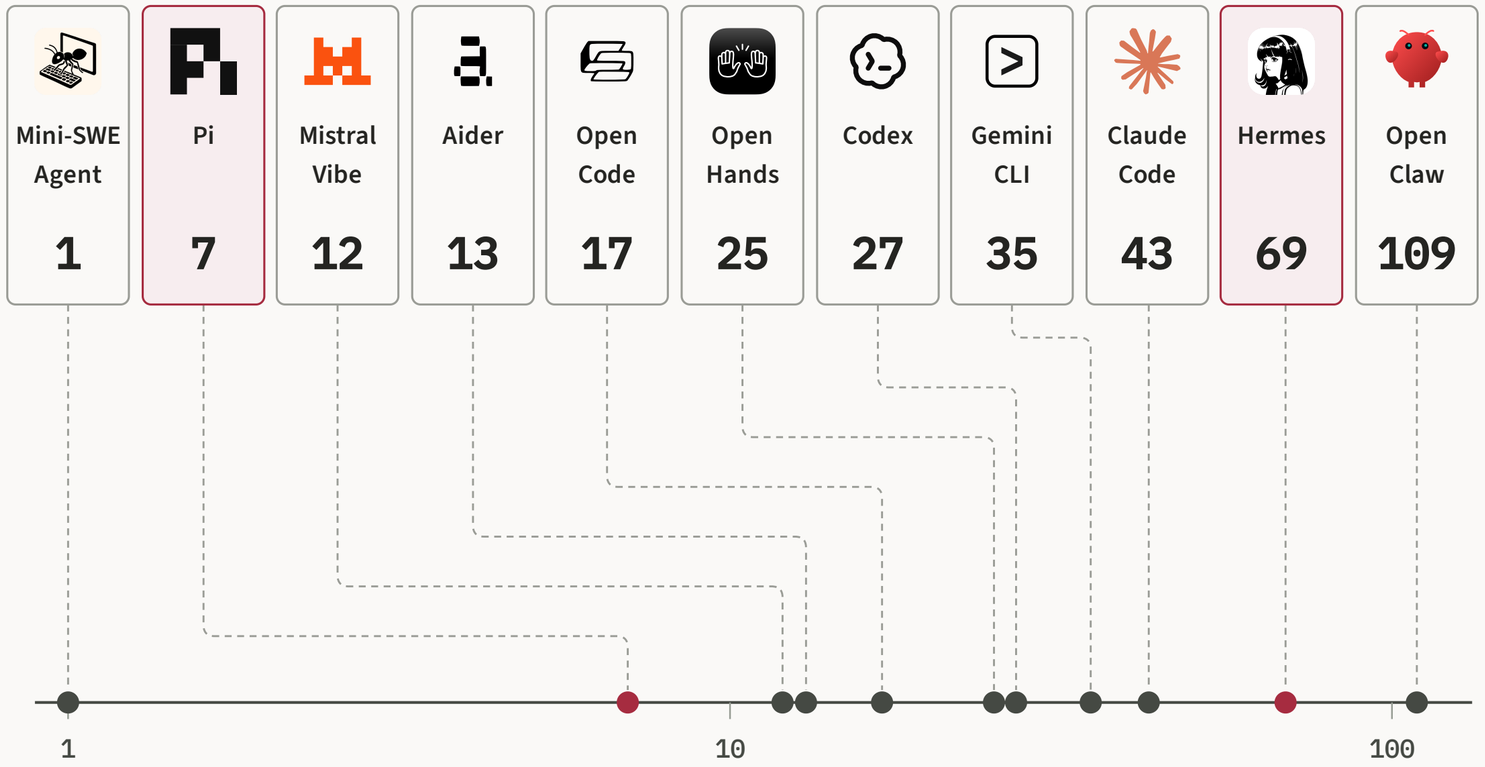
The harness is the part you can engineer.

Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §2.3, Table 1

TOPIC 1 / HARNESS ENGINEERING

Eleven harnesses, from one tool to 109

The same seven subsystems, built with tool surfaces that differ by two orders of magnitude.



Built-in tools per harness · logarithmic scale

Distinct built-in tools or edit formats, July 2026. Pi builds 7 and exposes 4 by default; Hermes scopes its 69 per platform.

KEY CONCEPTS

- **Minimal end:** Mini-SWE-Agent works through one bash tool; Pi exposes four by default.
- **Maximal end:** Hermes registers 69 tools and scopes them per platform; OpenClaw ships 109.
- **More code is not better results:** loop sophistication does not predict benchmark performance.
- **Most code goes elsewhere:** safety, user experience, extensibility and clients.

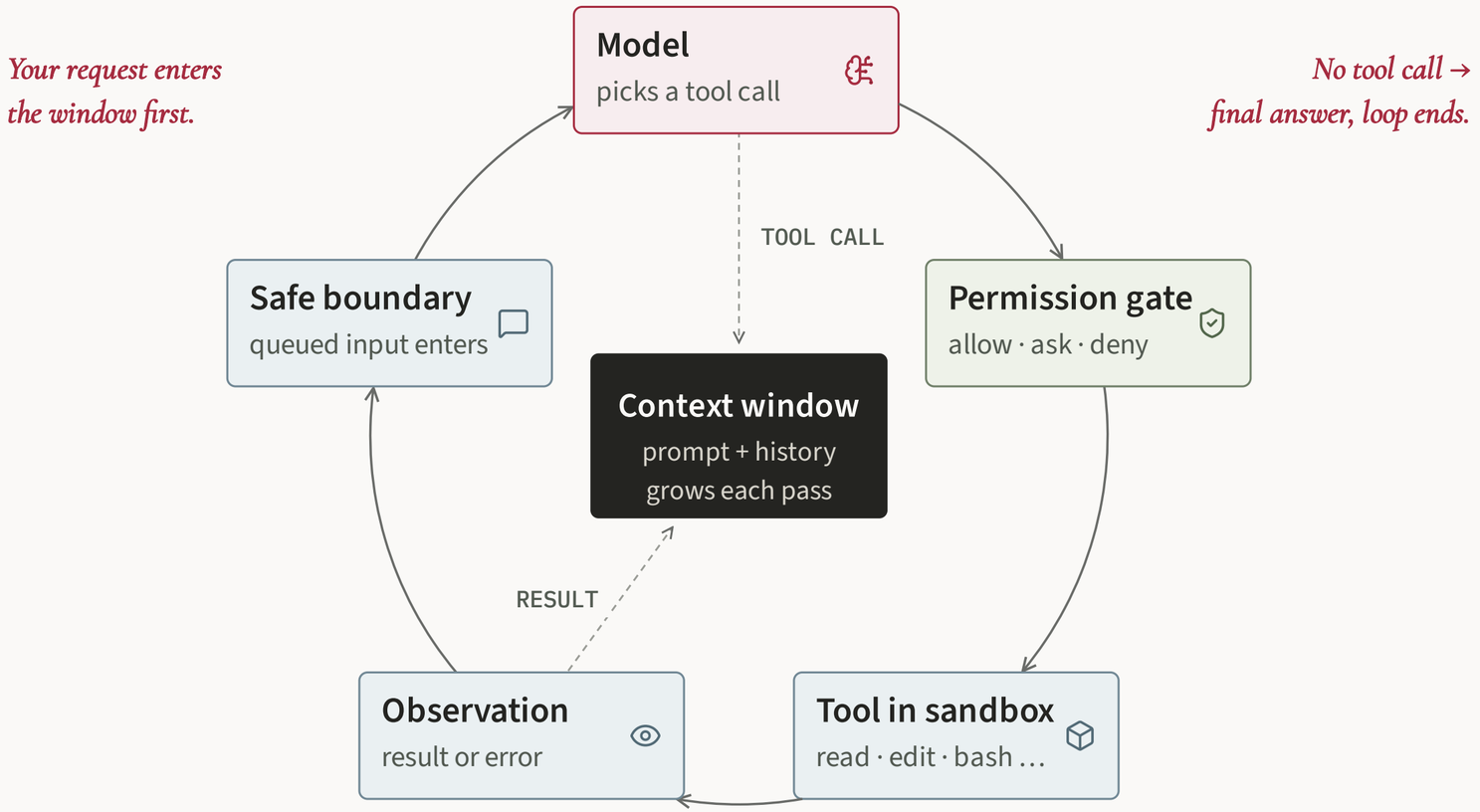
Tool count is a design philosophy, not a quality score.

Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §5 Table 4, §8.1 Fig. 3

TOPIC 1 / HARNESS ENGINEERING

The agent loop: decide, act, observe

One cycle repeats until the model answers in text instead of calling a tool.



Stop conditions differ: Pi has no turn cap; Hermes budgets 90 iterations and adds stop-guards.

KEY CONCEPTS

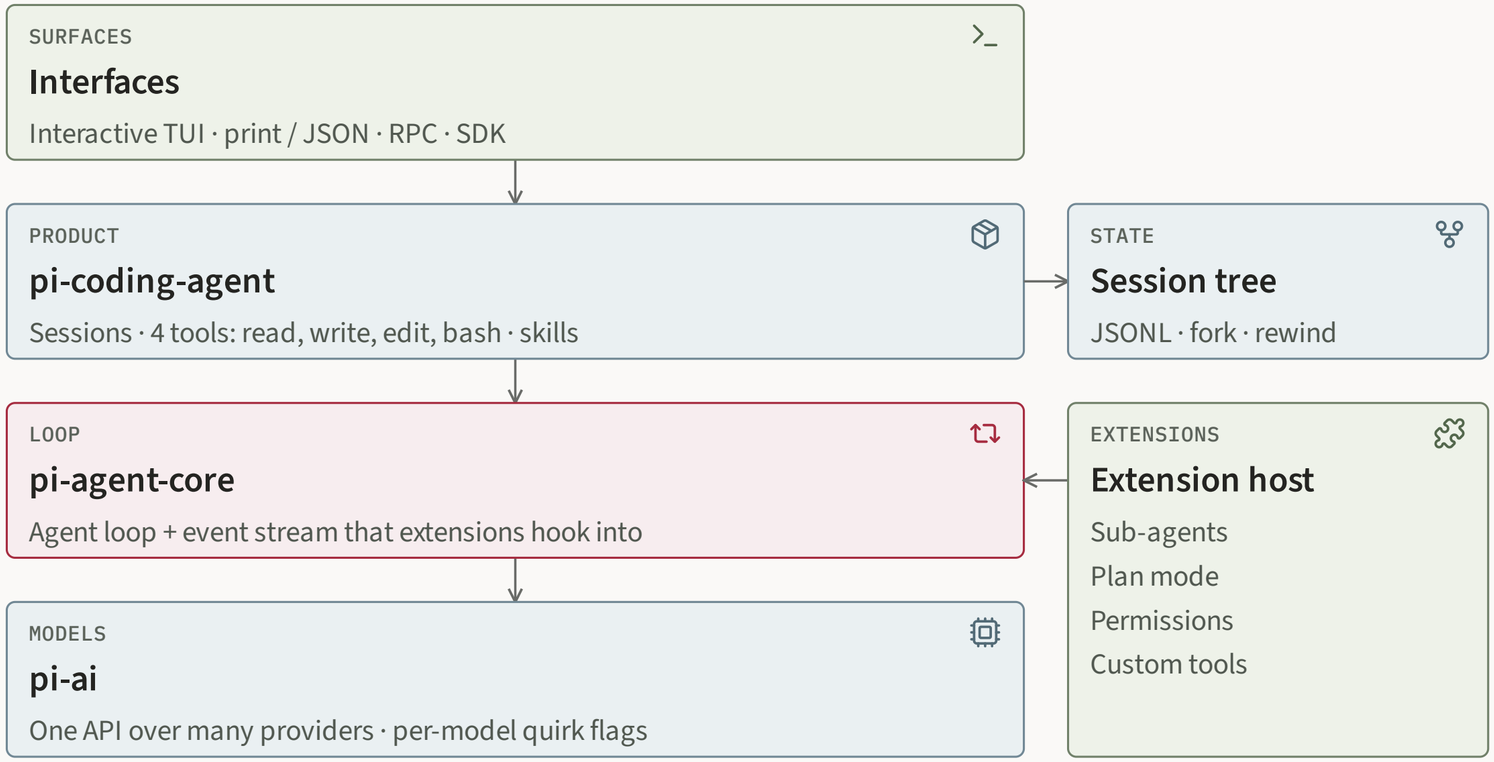
- **The context window accumulates state:** every pass writes into it.
- **Clockwise:** model → permission gate → tool → observation.
- **Steering input** waits for a safe boundary, never mid-tool-call.
- **Stop conditions differ:** no cap in Pi, a 90-step budget in Hermes.

A loop with a growing context — everything else manages that loop.

TOPIC 1 / HARNESS ENGINEERING

Pi: a minimal core, everything else an extension

Pi keeps the core small and moves safety, sub-agents and plan mode into extensions.



Safety is a documented absence in the core: permissions arrive as extensions.

KEY CONCEPTS

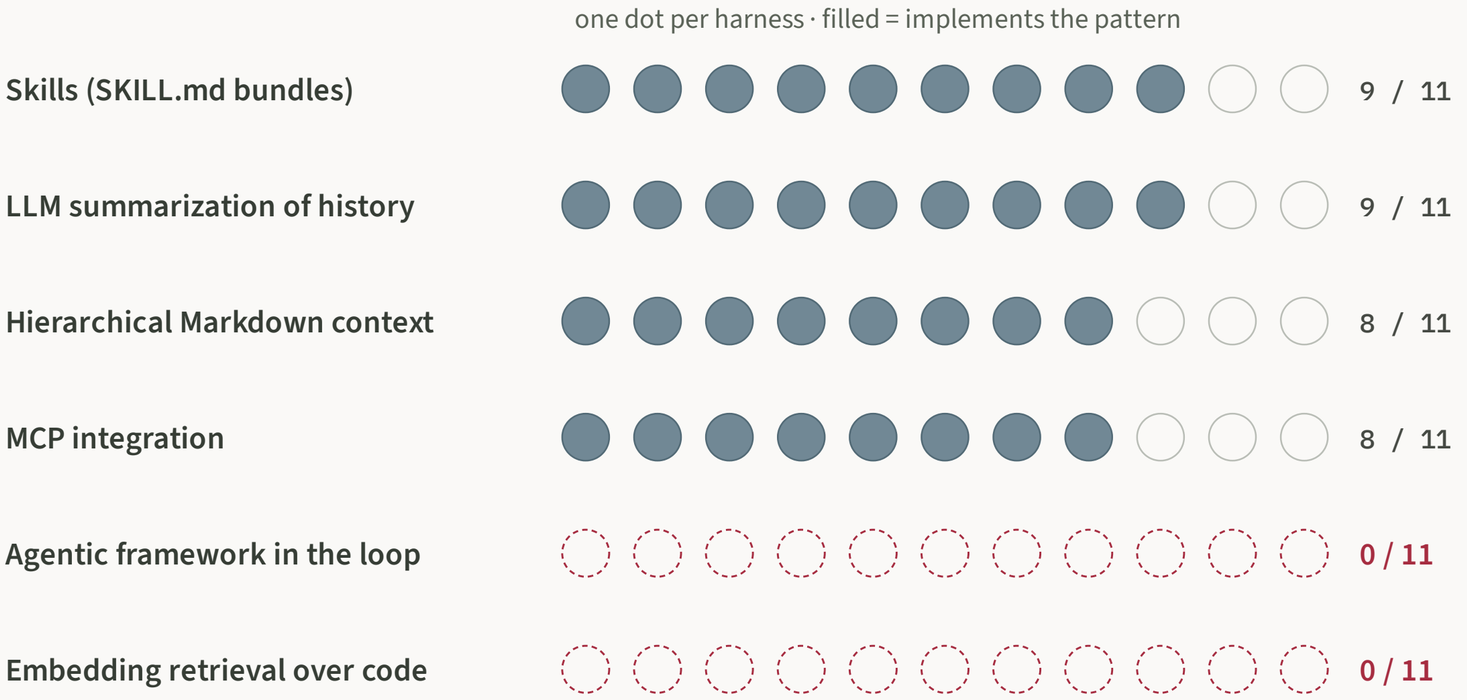
- **Four layers:** interfaces → product → loop → model gateway.
- **Session tree:** JSONL entries you can fork and rewind.
- **Extension host:** sub-agents, plan mode and permissions hook into the event stream.
- **Trade-off:** a small core, but safety depends on what you install.

Minimal core plus extensions is one coherent harness architecture.

TOPIC 1 / HARNESS ENGINEERING

What eleven harnesses share — and lack

Production harnesses converge on skills and context files, and all skip two popular technologies.



None of the eleven: hand-written loops and grep-style search instead.

Eleven production harnesses at July 2026 pins · source-level audit

KEY CONCEPTS

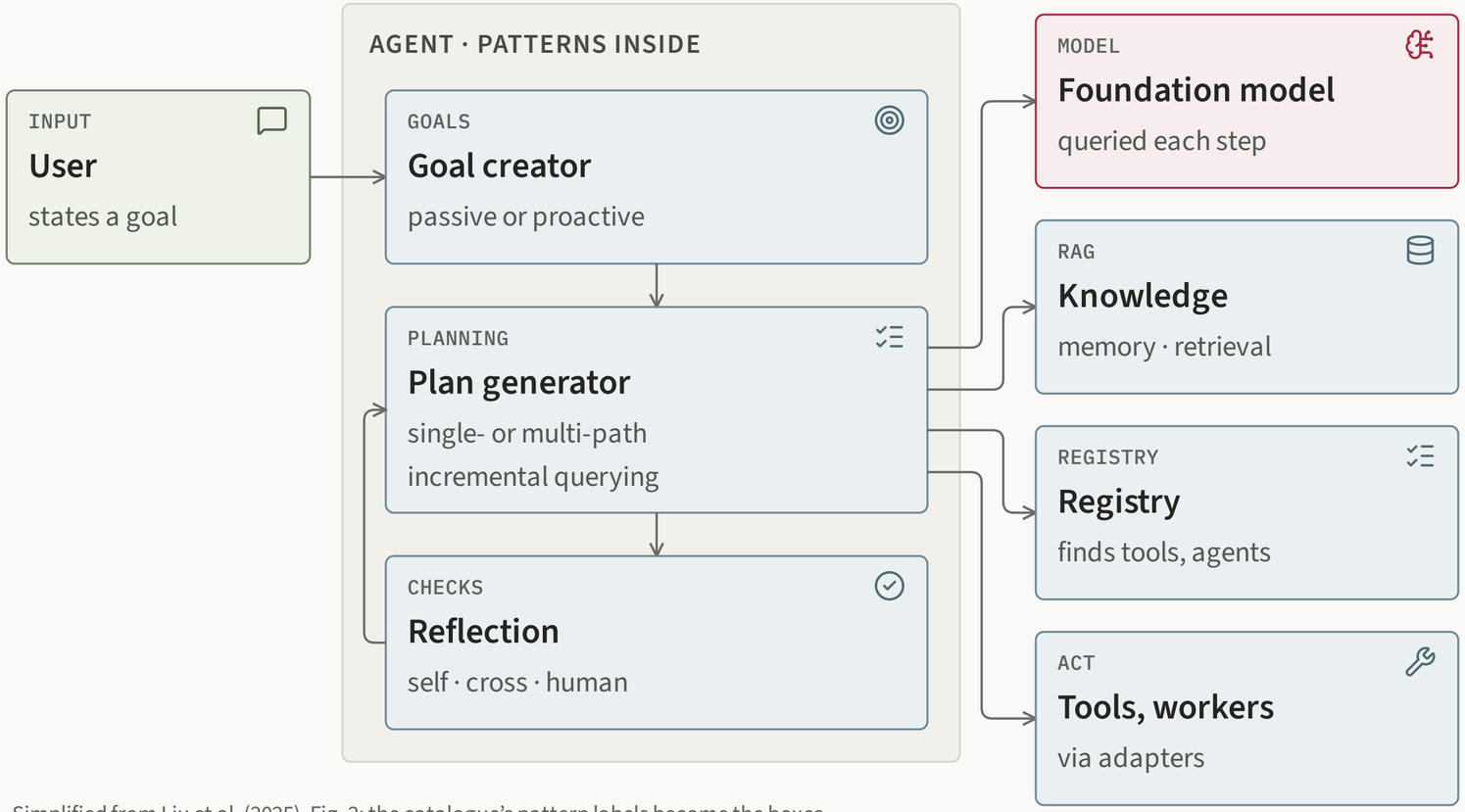
- **Shared:** skills, summarised history, Markdown context files, MCP.
- **Absent everywhere:** agentic frameworks and embedding search over code.
- **Instead:** hand-rolled async loops and deterministic search (grep, LSP).
- **Next:** a vocabulary for the agent inside the loop.

The field runs on simple loops and deterministic retrieval.

TOPIC 2 / DESIGN PATTERNS

A shared vocabulary for agent design

A systematic review of 57 studies distilled 18 architectural patterns for foundation-model agents.



Simplified from Liu et al. (2025), Fig. 2: the catalogue's pattern labels become the boxes.

KEY CONCEPTS

- **Inside the agent:** goal creation, planning, reflection.
- **Outside it:** the model, knowledge, a registry, tools and workers.
- **Each pattern** documents context, forces, solution, consequences, known uses.
- **Why it matters:** two teams can compare designs precisely.

Patterns let us name a design choice and its trade-off.

Liu et al. (2025), Agent Design Pattern Catalogue, JSS 220, §2–4, Fig. 2

TOPIC 2 / DESIGN PATTERNS

Eighteen patterns in eight families

Group the catalogue by the question each pattern answers.

Goal creation

- ○ Passive goal creator
- ○ Proactive goal creator

Prompts & knowledge

- ○ Prompt/response optimiser
- ○ RAG (retrieval)

Model querying

- ○ Incremental model querying
- ○ One-shot model querying

Plan generation

- ● Single-path plan generator
- ● Multi-path plan generator

Reflection

- ● Self-reflection
- ● Cross-reflection
- ● Human reflection

Cooperation

- ○ Role-based cooperation
- ○ Voting-based cooperation
- ○ Debate-based cooperation

Guardrails & tools

- ● Tool/agent registry
- ● Agent adapter
- ○ Multimodal guardrails

Evaluation

- ○ Agent evaluator

● **covered on the next three slides** ○ **see the catalogue for details**

KEY CONCEPTS

- **Eight families**, from goal creation to evaluation.
- **Most agents combine** several small patterns.
- **Deep dive on three:** planning, reflection, registry.
- **Coding harnesses skip RAG** and use grep and the language server instead.

Most agents combine many small patterns, not one big one.

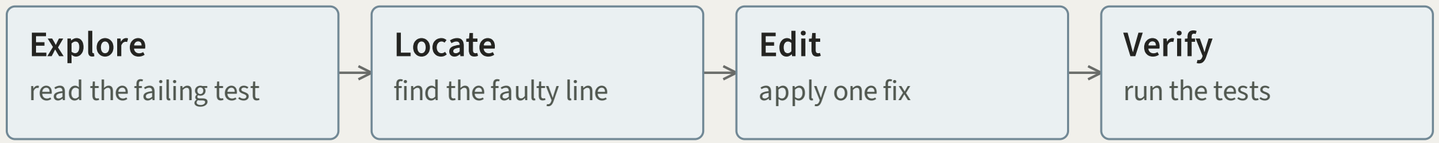
Liu et al. (2025), Agent Design Pattern Catalogue, JSS 220, Table 1 · Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §13.2

TOPIC 2 / DESIGN PATTERNS

Planning: one path or many?

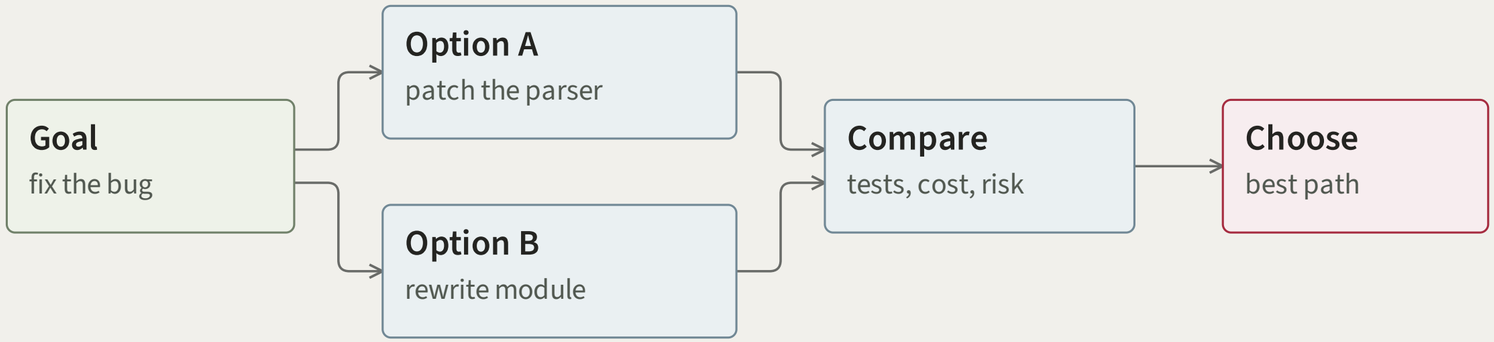
A plan generator decomposes a goal; the choice is how many options it keeps open.

SINGLE-PATH PLAN GENERATOR · ONE ORDERED LIST



Coding agents often keep this list visible as a todo tool;
a plan mode can draft it with read-only tools before any edit.

MULTI-PATH PLAN GENERATOR · OPTIONS AT EACH STEP



KEY CONCEPTS

- **Single path:** one ordered list — cheap, simple, explainable.
- **Multi-path:** options per step, compared before choosing.
- **Trade-off:** multi-path improves certainty but costs more model calls.
- **A visible plan** lets people correct it early.

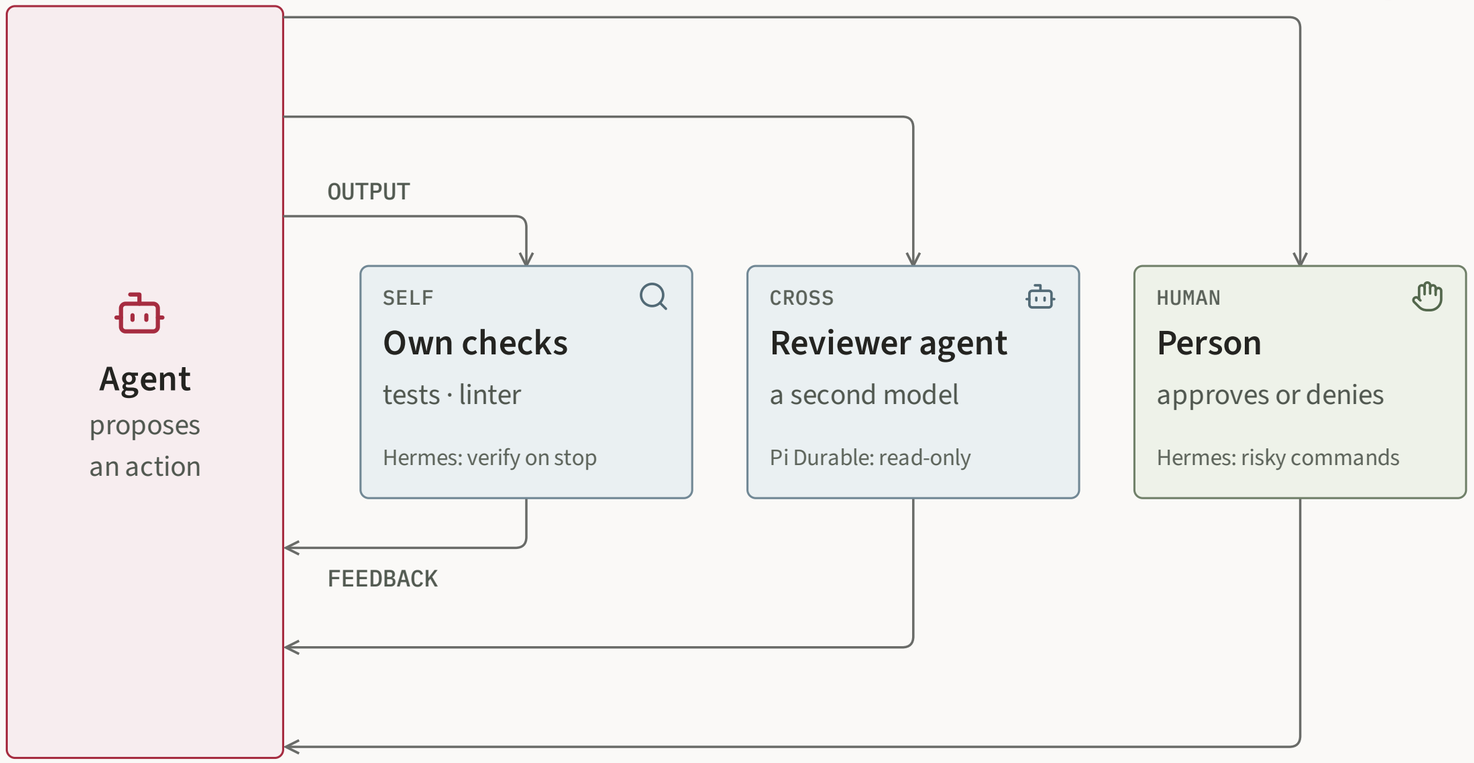
Most coding agents choose a single, visible plan.

TOPIC 2 / DESIGN PATTERNS

Reflection: who checks the work?

Reflection patterns differ by who gives the feedback: the agent, another agent, or a person.

Further out: slower, but more independent.



Liu et al. (2025): self-, cross- and human reflection. Examples from the Hermes and Pi Durable documentation.

KEY CONCEPTS

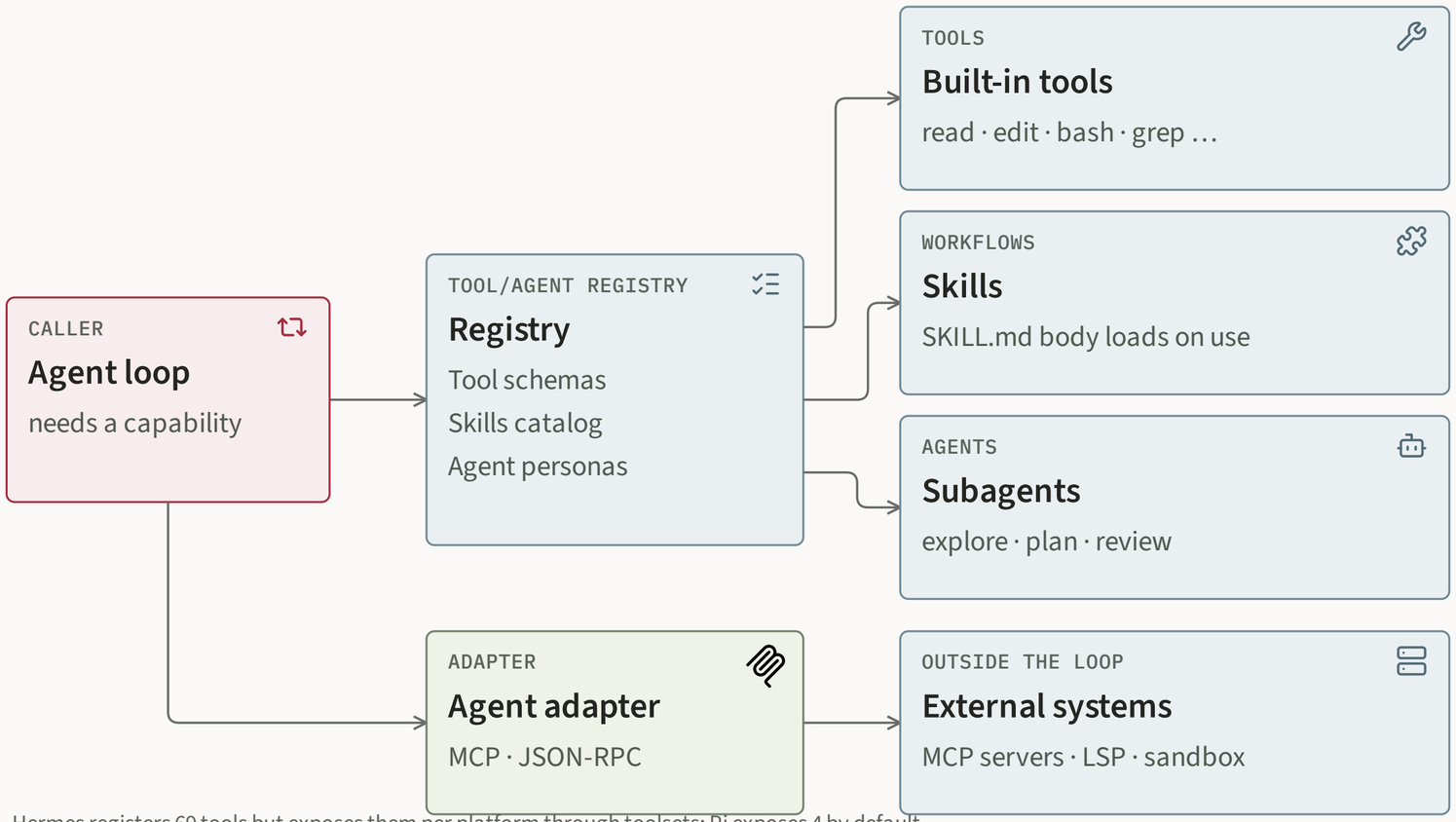
- **Self:** cheap and fast, limited by the agent's own blind spots.
- **Cross:** a second agent adds a perspective, and cost.
- **Human:** aligns with preferences, adds latency and contestability.
- **Real harnesses mix all three** at different moments.

Choose the reviewer by the cost of a wrong action.

TOPIC 2 / DESIGN PATTERNS

Registries and adapters: finding capabilities

A registry makes tools and agents discoverable; an adapter connects the loop to outside systems.



Hermes registers 69 tools but exposes them per platform through toolsets; Pi exposes 4 by default.

KEY CONCEPTS

- **Registry:** tool schemas, skills and agent personas in one place.
- **Load on use:** skills and agents enter the context only when needed.
- **Adapter:** MCP or JSON-RPC to servers, language servers and sandboxes.
- **Scale:** Hermes registers 69 tools but shows each platform a subset.

Registries keep the loop small while capabilities grow.

Liu et al. (2025), Agent Design Pattern Catalogue, JSS 220, §4.16–4.17 · Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §8

TOPIC 2 / DESIGN PATTERNS

Choosing patterns: the decision model

The catalogue's decision model asks one question per design problem.

DECISION	YES →	NO →
Capture the user's environment?	Proactive goal creator	Passive goal creator
Use an external data store?	Retrieval-augmented generation	Model + tools only
Query the model several times?	Incremental model querying	One-shot model querying
Several options per plan step?	Multi-path plan generator	Single-path plan generator
Multiple agents collaborate?	Cross-reflection, cooperation	Self- or human reflection
Use external tools or agents?	Tool/agent registry, adapter	—
Assess agent performance?	Agent evaluator	—

KEY CONCEPTS

- **One question per row**, two answers.
- **Each answer has consequences** — benefits and costs.
- **A design** is a coherent set of answers.
- **Next:** how do we judge whether the choices are good?

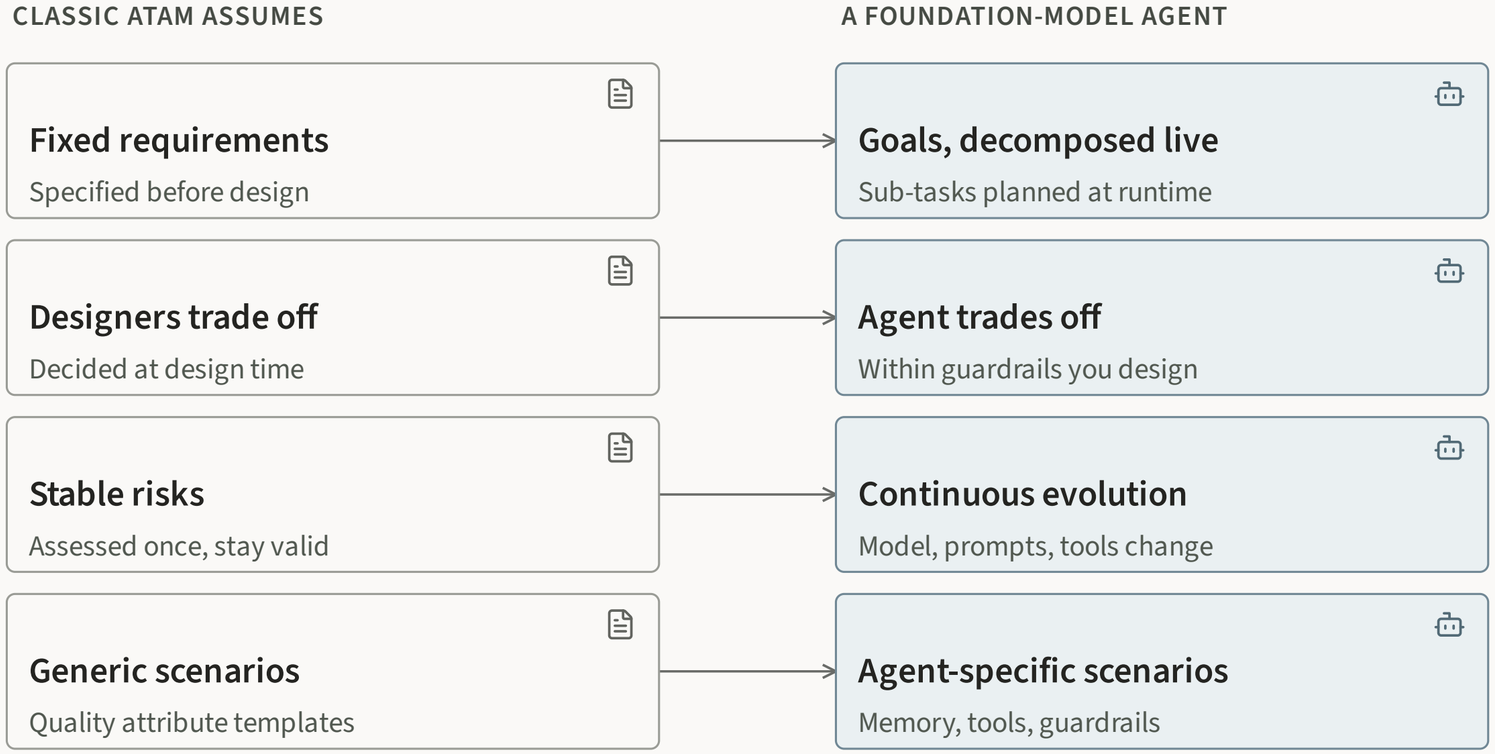
A pattern is chosen by a question, not by fashion.

Liu et al. (2025), Agent Design Pattern Catalogue, JSS 220, §5.1, Fig. 19

TOPIC 3 / ARCHITECTURE EVALUATION

Why architecture evaluation must change

Classic scenario-based methods assume fixed requirements; agents decide much of their architecture at runtime.



Guardrail design becomes the principal architectural control point.

KEY CONCEPTS

- **Goals** are decomposed into sub-tasks while the agent runs.
- **The agent makes trade-offs** — inside guardrails we design.
- **Model, prompts and tools** change after evaluation.
- **New scenarios** are needed for memory, tools and guardrails.

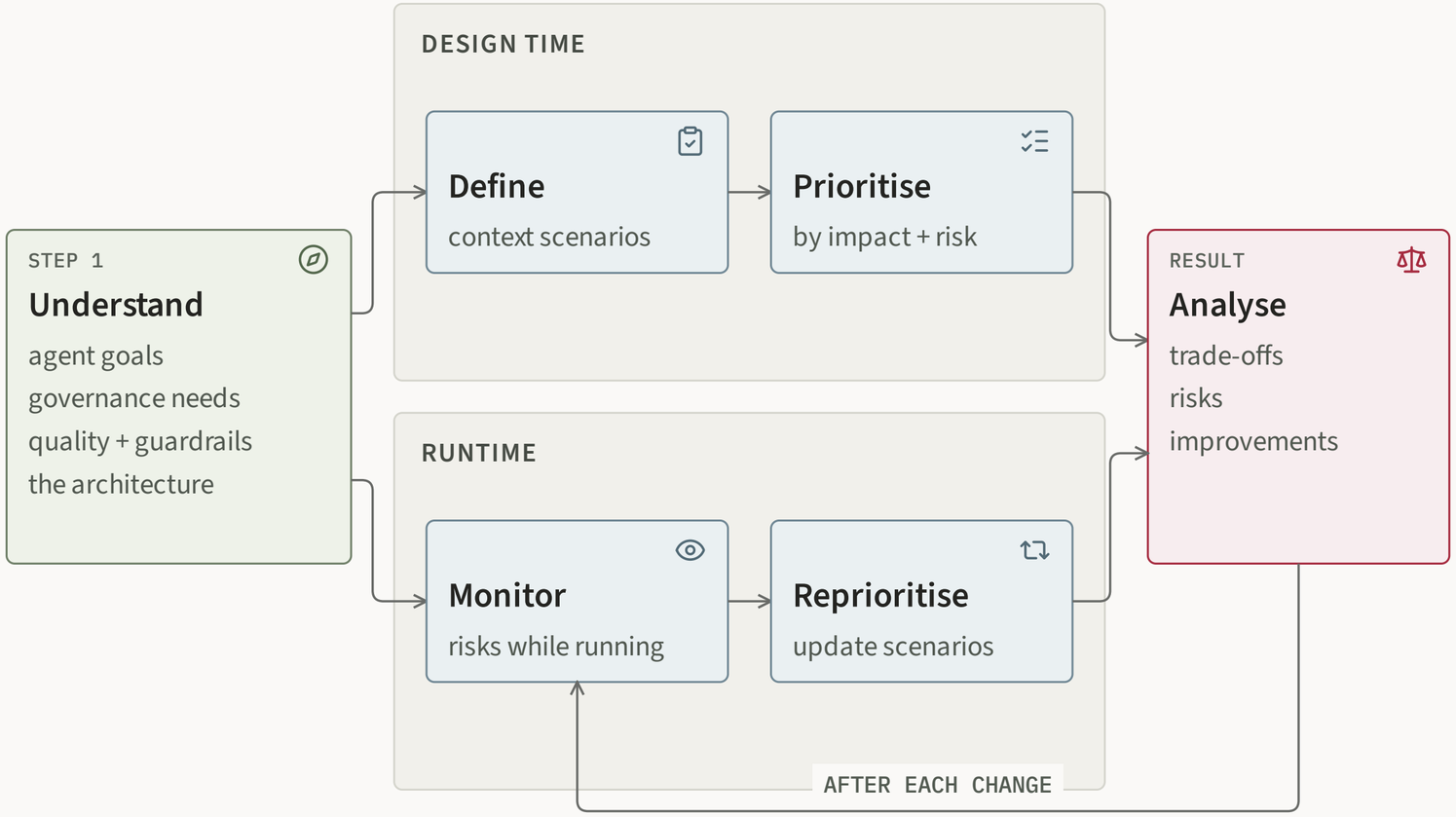
Guardrails become the main architectural control point.

Lu et al. (2026), AgentArcEval, JSS 232, §1

TOPIC 3 / ARCHITECTURE EVALUATION

AgentArcEval: the process

An ATAM-based method that evaluates agent architecture at design time and keeps re-evaluating at runtime.



KEY CONCEPTS

- **Understand first:** goals, governance, quality and guardrail requirements, the architecture.
- **Design time:** define and prioritise context scenarios.
- **Runtime:** monitor risks and reprioritise scenarios.
- **Output:** trade-offs, risks, improvements.

Agent architecture evaluation never really ends.

Lu et al. (2026), AgentArcEval, JSS 232, §4, Fig. 2

TOPIC 3 / ARCHITECTURE EVALUATION

A quality scenario in six parts

Each scenario follows source → stimulus → environment → artefact → response → measure.

An accuracy scenario from the Luna tax-copilot case

Each part is checkable: the measure says what "good" looks like.

SOURCE	STIMULUS	ARTEFACT	RESPONSE	MEASURE
Tax expert	**Query**	**Pipeline**	**Answer**	**≥ 95%**
a tax professional	capital gains tax on a rental property sale	context engine planner retriever	latest rules, advice in context	accurate answers and correct references

Environment

Tax season: accurate recommendations are key

Template: source → stimulus → environment → artefact → response → response measure (Lu et al., §5).

Values: the Luna accuracy context scenario (Lu et al., §6.4), shortened.

KEY CONCEPTS

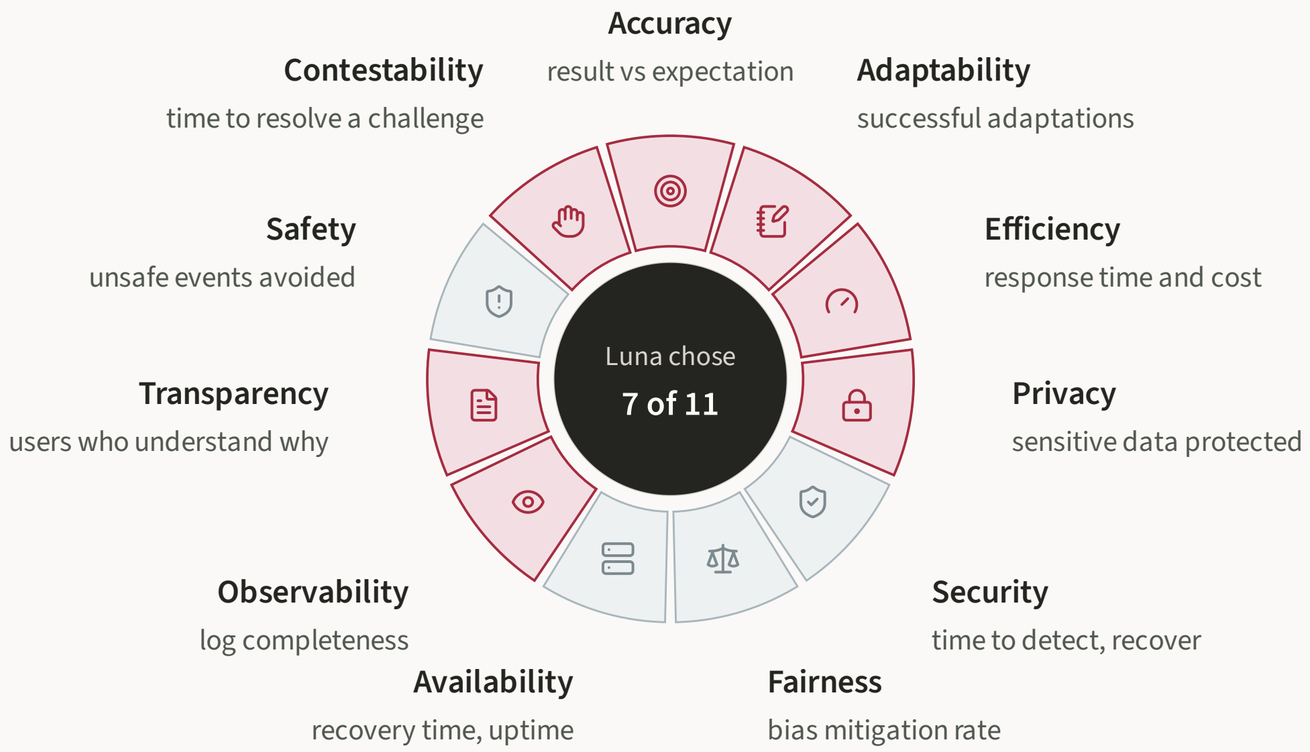
- **Source and stimulus:** who triggers what.
- **Artefact:** the part of the architecture under test.
- **Environment:** the conditions that make it hard.
- **Measure** makes the scenario checkable.

A scenario is only useful if its measure can fail.

TOPIC 3 / ARCHITECTURE EVALUATION

Eleven general scenarios and their measures

The catalogue gives one starter scenario per quality attribute; teams adapt and prioritise them.



KEY CONCEPTS

- **Eleven attributes**, from accuracy to contestability.
- **Each comes with a measure** a team can track.
- **Prioritising** is where the domain enters.
- **Luna tax copilot** chose 7 of the 11.

Prioritising scenarios is where the domain enters the evaluation.

Lu et al. (2026), AgentArcEval, JSS 232, §5–6

TOPIC 3 / ARCHITECTURE EVALUATION

Scenarios as a lens on real harnesses

The same scenario catalogue exposes where real harnesses put their architectural effort.

SCENARIO	PI	PI DURABLE	HERMES
Safety	None in the core, by design; extensions add it	Hooks can block a tool call; approvals stored as memos	Approval module; a hard floor of blocked commands
Availability	You restart and ask it to continue	resume() continues open tasks after a crash	Long-running gateway process
Observability	Full JSONL session tree on disk	Every step committed; clients watch live	SQLite sessions with full-text search
Efficiency	Compacts when only a 16k reserve is left	Compacts in the background	Compacts at 50%; rotates the session
Adaptability	Extensions reload without a build step	Extensions replaced while running	Agent writes and patches its own skills

KEY CONCEPTS

- **Rows** are AgentArcEval attributes; cells are architectural mechanisms.
- **Pi** optimises for a person at a terminal.
- **Pi Durable** optimises for availability.
- **Hermes** invests in safety policy and self-improvement.

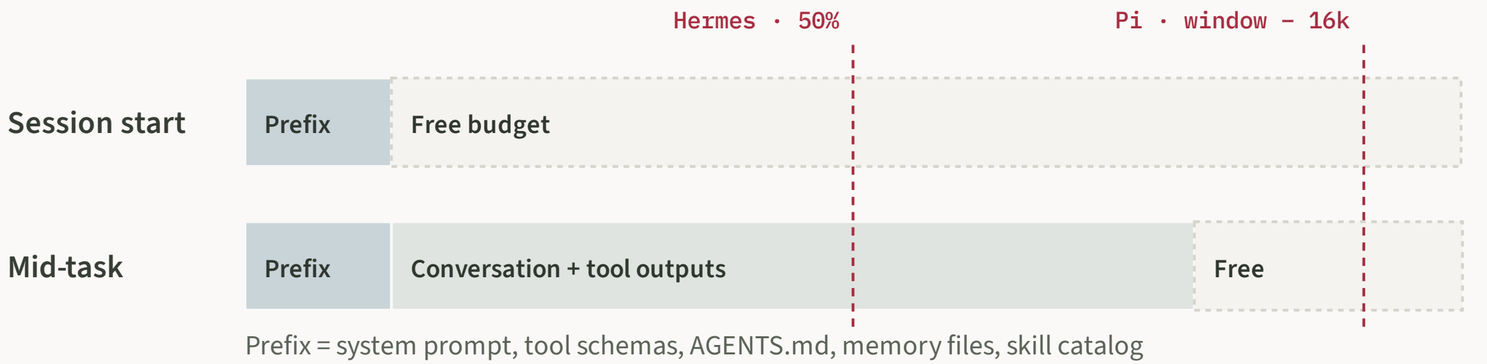
A scenario catalogue makes the trade-offs between harnesses visible.

Lu et al. (2026), AgentArcEval, JSS 232 · our mapping from Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, Earendil (2026), Pi Durable, earendil.com/posts/pi-durable

TOPIC 4 / CONTEXT ENGINEERING

Context is a budget

Context engineering decides what task-relevant information the model has — and what it does not.



Prefix = system prompt, tool schemas, AGENTS.md, memory files, skill catalog

Summarise in place
Pi: older turns become a structured checkpoint; the full history stays on disk

Summarise in the background
Pi Durable: the summary is built while the conversation keeps going

Rotate the session
Hermes: compaction closes the session and opens a child; nothing is deleted

Proportions illustrative for a 200k-token window. Pi reserves 16,384 tokens by default; Hermes triggers at 50%.

KEY CONCEPTS

- **Prompt engineering:** how a task is described.
- **Context engineering:** what information the model can access.
- **The window fills** with every tool output.
- **Compaction** frees space — at different thresholds and in different ways.

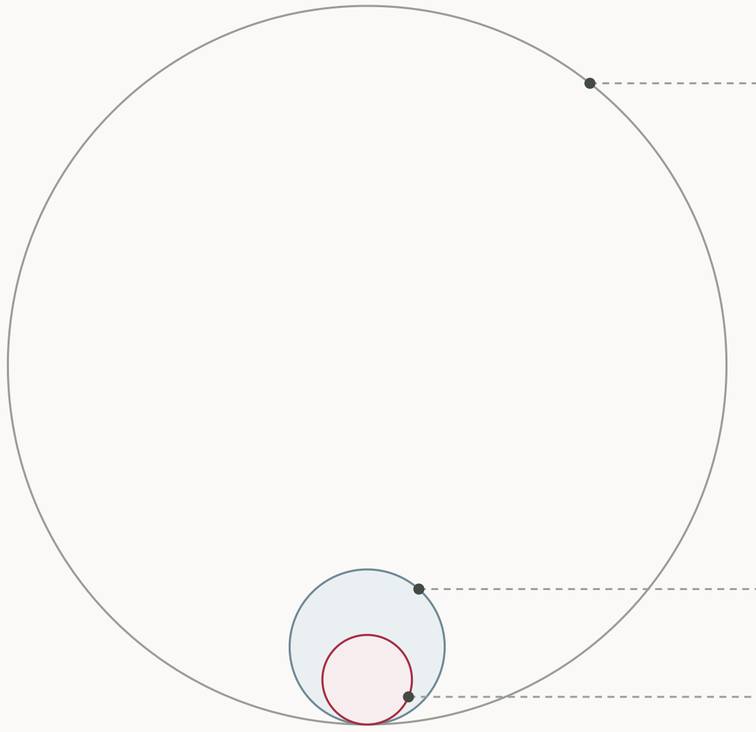
Every token in the window competes for the model's attention.

Mohsenimofidi et al. (2026), Context Engineering for AI Agents in OSS, MSR '26, §1 · Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §7

TOPIC 4 / CONTEXT ENGINEERING

Who actually writes context files?

An empirical study scanned 10,000 popular GitHub projects for AI context files.



Circle areas to scale · Mohsenimofidi et al. (2026), §3

10,000 popular GitHub repositories scanned

FORMATS FOUND

- Copilot instructions
- CLAUDE.md
- AGENTS.md
- GEMINI.md

453 commits to these 155 files show how they change over time.

466 had an AI context file · about 5%

155 AGENTS.md files analysed in depth

KEY CONCEPTS

- **Formats:** Copilot instructions, CLAUDE.md, AGENTS.md, GEMINI.md.
- **Adoption is early:** only 5% of mature projects.
- **AGENTS.md** is the tool-agnostic convention, so it is studied in depth.
- **Most common pair:** AGENTS.md with CLAUDE.md.

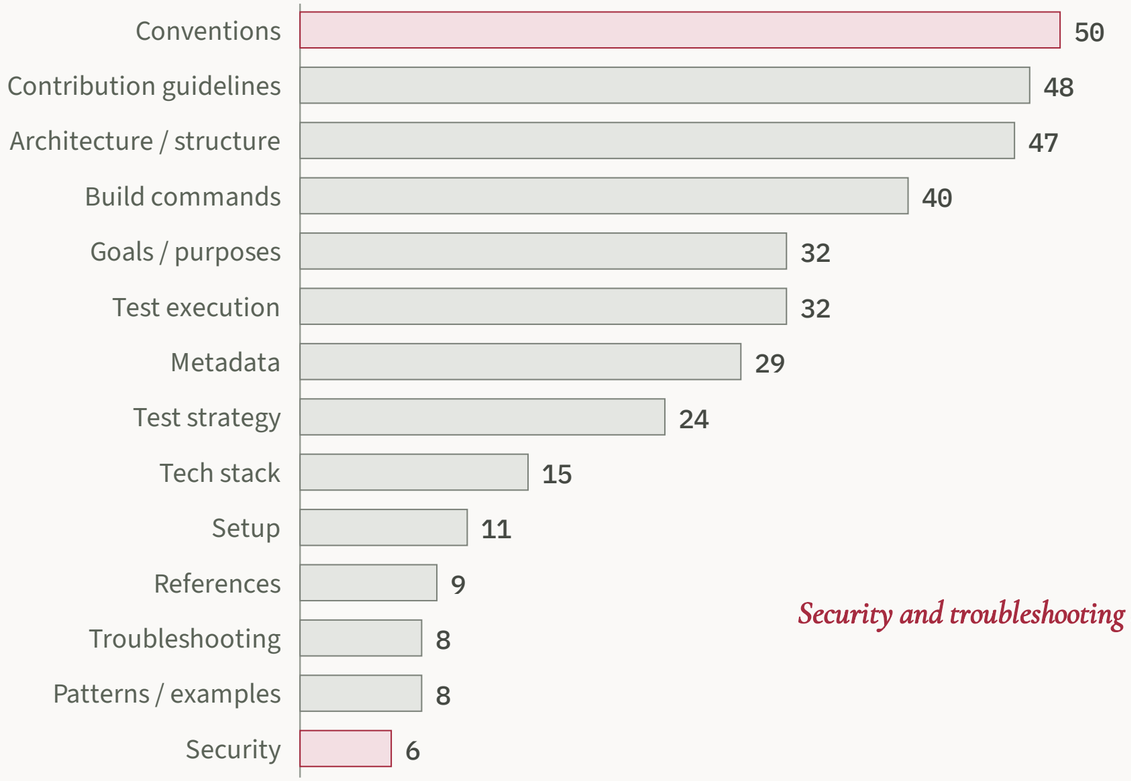
Context engineering is real practice, but still early.

TOPIC 4 / CONTEXT ENGINEERING

What goes into AGENTS.md

Projects describe conventions, contribution rules and architecture — rarely security.

Level-1 and level-2 headings per category · 155 AGENTS.md files



Security and troubleshooting are rare.

KEY CONCEPTS

- **Top three:** conventions, contribution guidelines, architecture.
- **Then:** build and test commands — what an agent must run.
- **Rare:** troubleshooting and security.
- **No shared structure** yet across projects.

Teams document how to work here, not how to stay safe.

Mohsenimofidi et al. (2026), Context Engineering for AI Agents in OSS, MSR '26, §4.2, Table 1

TOPIC 4 / CONTEXT ENGINEERING

Five ways to phrase an instruction

The same convention can describe, order, forbid, explain or set a condition.

STYLE 1
Descriptive
states a fact

"This project uses the Linux Kernel Style Guideline."

STYLE 2
Prescriptive
gives an order

"Follow the existing code style and conventions."

STYLE 3
Prohibitive
sets a boundary

"Never commit directly to the main branch."

STYLE 4
Explanatory
adds the reason

"Avoid hard-coded waits to prevent timing issues in CI environments."

STYLE 5
Conditional
encodes a situation

"If you need to use reflection, use ReflectionUtils APIs."

Examples quoted from the Conventions sections studied by Mohsenimofidi et al. (§4.2); styles are often mixed in one file.

KEY CONCEPTS

- **Descriptive** states facts; **prescriptive** gives orders.
- **Prohibitive** sets boundaries — "never …".
- **Explanatory** adds the reason; **conditional** a situation.
- **Open question:** which style changes agent behaviour most?

How you phrase context is a design choice, not a detail.

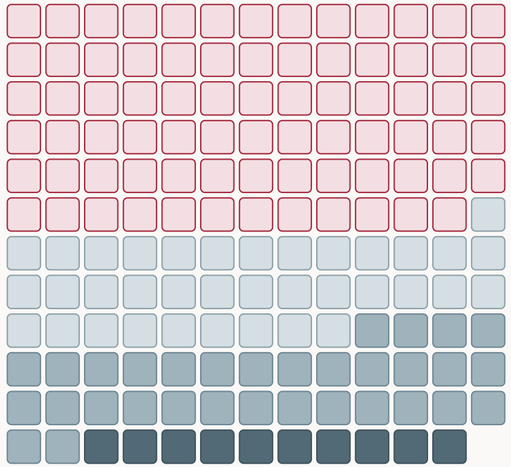
TOPIC 4 / CONTEXT ENGINEERING

Context files are maintained artifacts

Half the files never change; active ones are tuned instruction by instruction.

How often was each file changed?

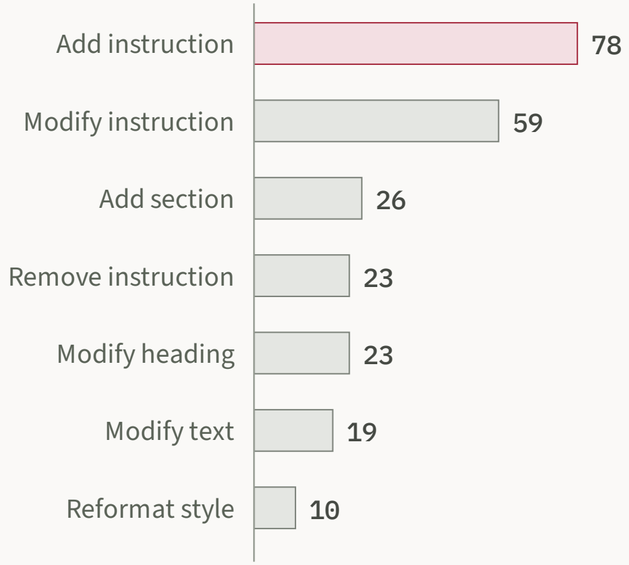
one square per AGENTS.md file · 155 files



Never	77	50%
Once	36	23%
2–7 times	32	21%
10+ times	10	6%

What changed? (169 commits)

the 10 files with ≥10 commits



Mostly fine-tuning: instructions are added and reworded, sections rarely removed.

Mohsenimofidi et al. (2026), §4.3, Table 2. A commit can belong to more than one category.

KEY CONCEPTS

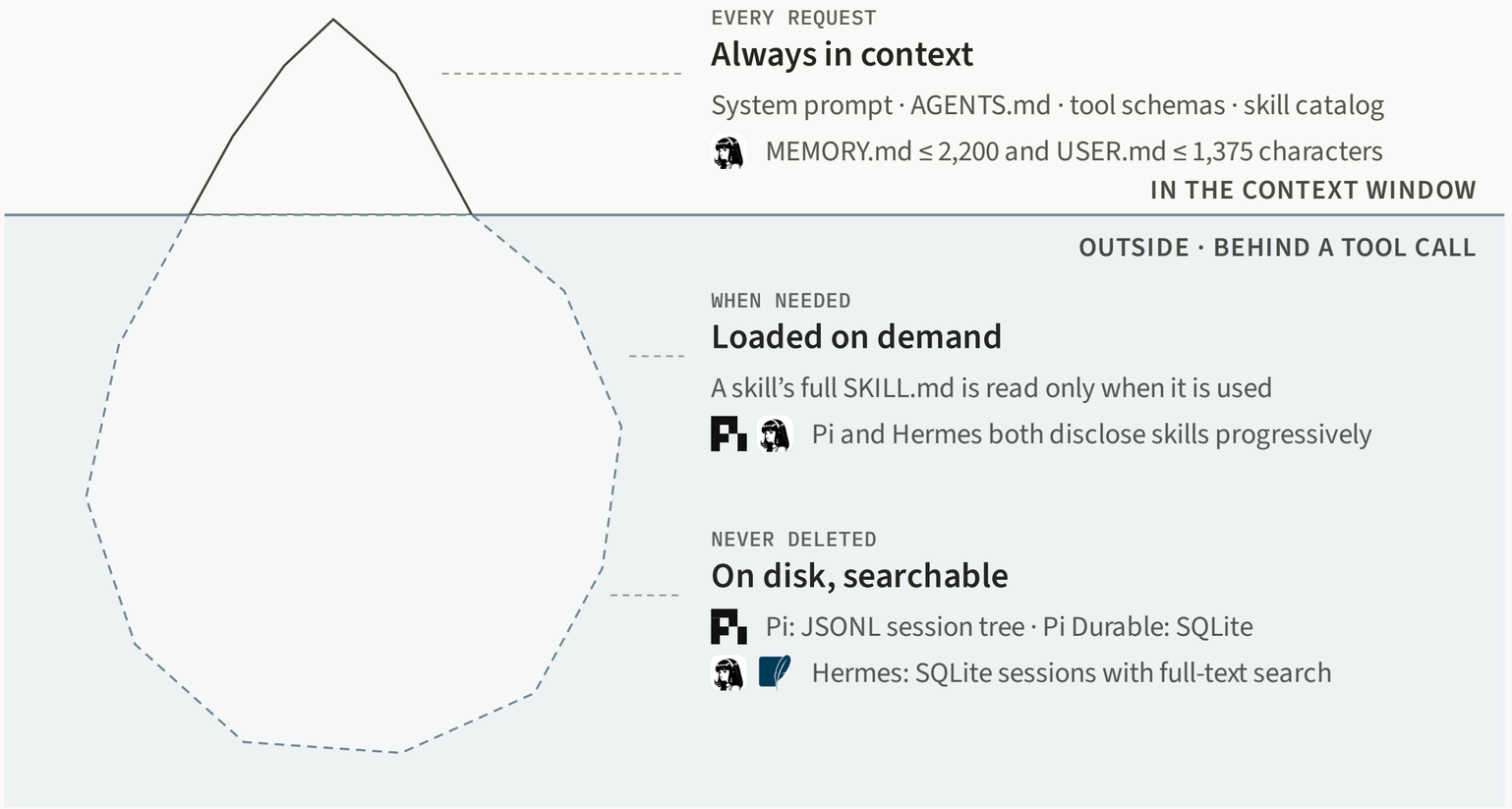
- **50% unchanged** since they were added.
- **Active files:** mostly add or reword instructions.
- **Few removals:** files tend to grow.
- **Treat them like code:** versioned, reviewed, tested.

Context files co-evolve with the code they describe.

TOPIC 4 / CONTEXT ENGINEERING

What stays in context, what waits on disk

Harnesses keep the always-loaded context small and move the rest behind a tool call.



KEY CONCEPTS

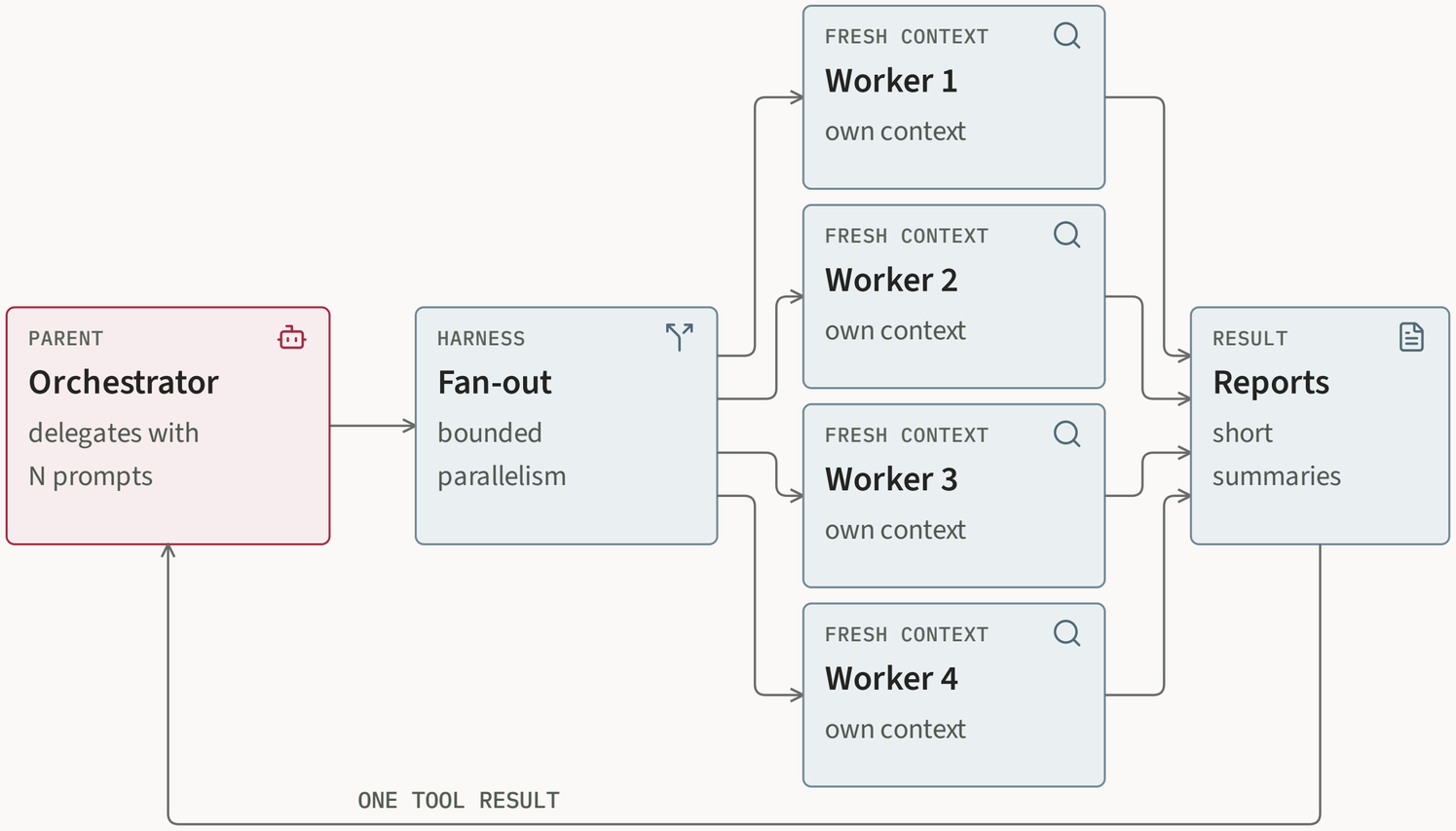
- **Always loaded:** instructions, schemas, a skill catalog, small memory files.
- **On demand:** a skill's full text only when it is used.
- **On disk:** full history, searchable but out of the window.
- **Hermes caps memory** at a few thousand characters on purpose.

Load context just in time, not just in case.

TOPIC 4 / CONTEXT ENGINEERING

Subagents are context engineering

A subagent spends tokens exploring in its own context and returns one short report.



KEY CONCEPTS

- **The orchestrator** delegates several prompts at once.
- **Workers** start with a fresh context and keep their search noise.
- **The parent keeps** only the call and the reports.
- **Hermes** runs one child by default; in Pi Durable a subagent is a conversation owned by a tool call.

The harness decides what the orchestrator never sees.

TOPIC 5 / FROM SESSIONS TO ALWAYS-ON AGENTS

From session agent to always-on agent

The same loop can run per session, survive crashes, or run all the time — each step adds state and responsibility.

Same loop at the core.
What grows: state, triggers and reach.

SESSION AGENT
Pi coding agent

a person starts it
state: JSONL session tree
crash: ask it to continue
reached from a terminal

DURABLE HARNESS · LIBRARY
Pi Durable

+ crash-safe state

your application submits work
state: a checkpoint per step
crash: resume() continues
experimental, beside Pi 1.0

ALWAYS-ON AGENT
Hermes Agent

+ triggers and memory

chats and schedules start it
state: SQLite + Markdown memory
writes and patches its skills
reached from chat apps:

Our framing of three points on a spectrum. Hermes (Nous Research) is a separate project, not built on Pi.

KEY CONCEPTS

- **Pi:** a person at a terminal starts and supervises each run.
- **Pi Durable:** a library for apps whose agents must survive restarts.
- **Hermes:** started by chats and schedules, not only by people.
- **What grows:** state, triggers and the surfaces that reach the agent.

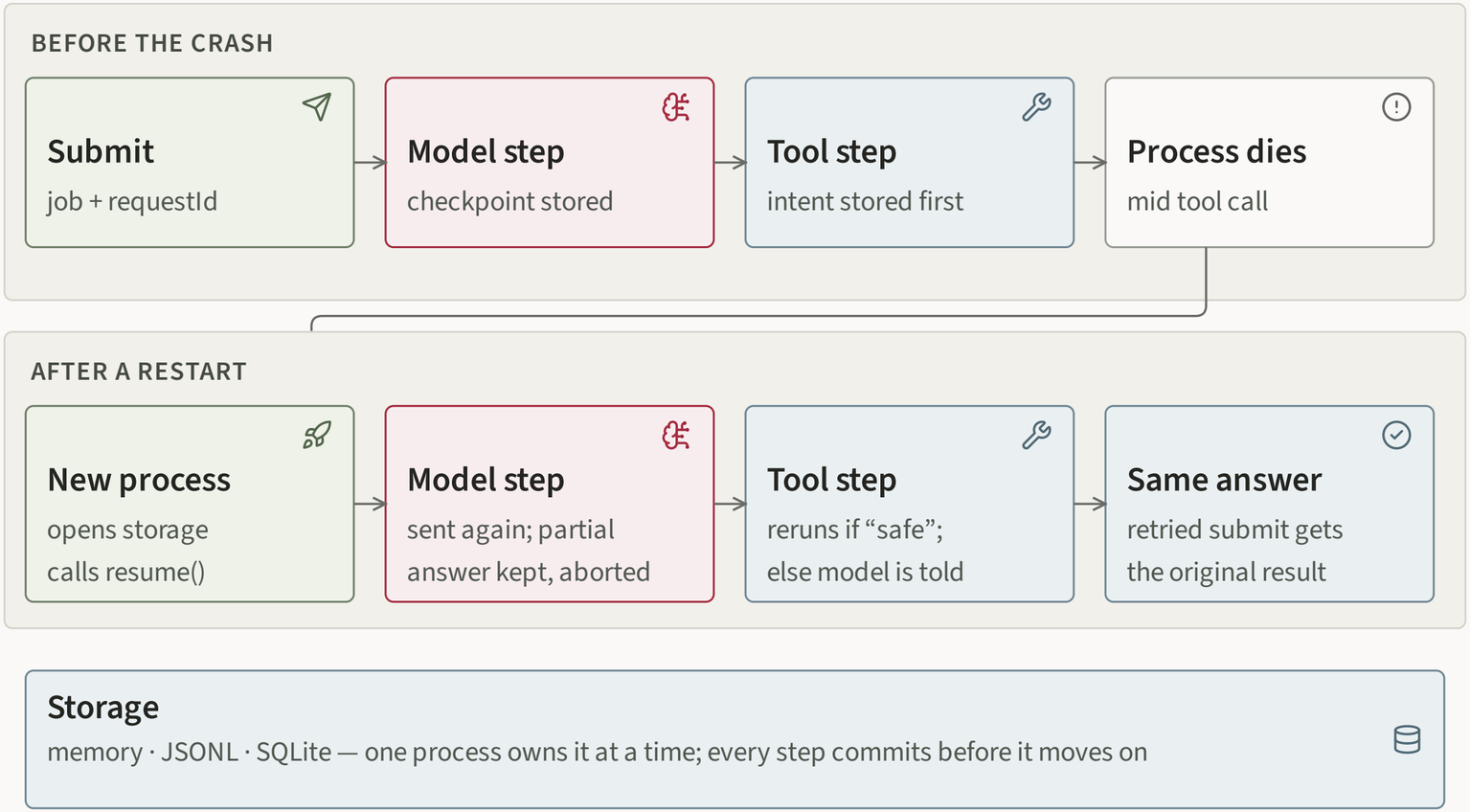
Always-on is an architectural decision, not a feature flag.

Zechner (2025); Pi docs, github.com/earendil-works/pi · Earendil (2026), Pi Durable, earendil.com/posts/pi-durable · Nous Research, Hermes Agent docs; Alejandro AO (2026), Hermes architecture

TOPIC 5 / FROM SESSIONS TO ALWAYS-ON AGENTS

Pi Durable: surviving a crash

Every step is a task that stores a checkpoint before it moves on.



KEY CONCEPTS

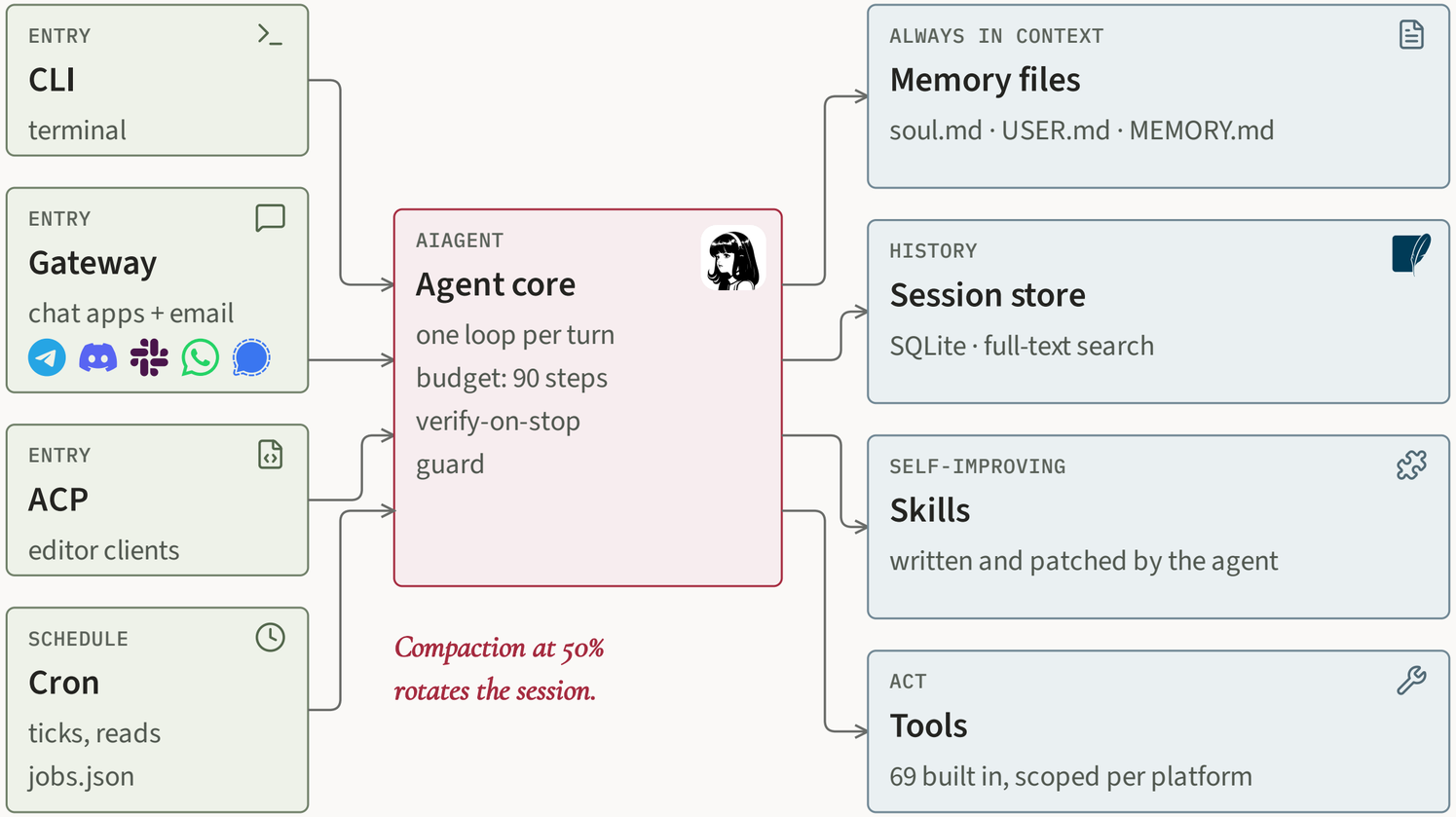
- **Checkpoint first:** a tool call’s intent is stored before it runs.
- **After a restart**, a new process resumes open tasks.
- **Replay rule:** only tools marked safe rerun; otherwise the model is told.
- **requestId** makes a submission exactly-once.

Durability means deciding, per tool, what may happen twice.

TOPIC 5 / FROM SESSIONS TO ALWAYS-ON AGENTS

Hermes: an always-on agent

One agent core, reached through a CLI, chat gateways and editors, and started by a scheduler.



KEY CONCEPTS

- **Entry points:** CLI, a long-running gateway, ACP clients.
- **Cron** starts work without a person in the loop.
- **Memory files** are always in context; sessions live in SQLite.
- **Self-improving:** the agent writes and patches its own skills.

An always-on agent is a service, with a service's responsibilities.

Nous Research, Hermes Agent docs; Alejandro AO (2026), Hermes architecture · Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §7–10

TOPIC 5 / FROM SESSIONS TO ALWAYS-ON AGENTS

What always-on costs

More autonomy moves risk from the person at the terminal into the architecture.

	PI	PI DURABLE	HERMES
Started by	A person, per session	Your application	People, chats and schedules
State	JSONL session tree	Checkpoints in SQLite, JSONL or memory	SQLite sessions + Markdown memory
After a crash	You ask it to continue	resume() picks up open tasks	History persists in SQLite
Isolation	None by design	Pluggable execution environments	Six pluggable execution backends
Watch out for	Safety depends on installed extensions	Experimental API; one process owns the storage	One user lost 69% of a 12-hour session's tokens to replayed history

KEY CONCEPTS

- **Each step right** adds state that must stay consistent.
- **Unattended runs** need durable checkpoints and replay rules.
- **Long-running processes** fail in new ways: leaks, replays, fragmentation.
- **Field reports** are evidence too: read the issue tracker.

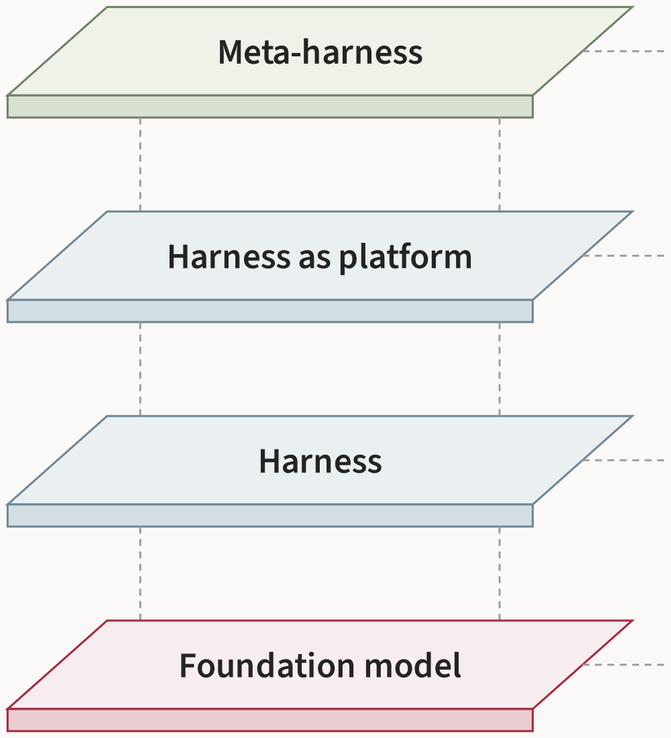
Every hour an agent runs unattended is an hour of architecture to own.

Zechner (2025); Pi docs, github.com/earendil-works/pi · Earendil (2026), Pi Durable, earendil.com/posts/pi-durable · Hermes issue #5563

TOPIC 5 / FROM SESSIONS TO ALWAYS-ON AGENTS

The platform turn

Harnesses are becoming platforms, and frameworks are becoming harnesses.



META-HARNESS · ORCHESTRATES HARNESSES

Omnigent: adapters for 23 harnesses, one API, sandbox and policy above them

DISTRIBUTION

SDKs · libraries like Pi Durable

ACP servers · skill marketplaces

RUNTIME · LOOP + SEVEN SUBSYSTEMS

The eleven harnesses of the study, for example:

CAPABILITY

Co-evolves with its harness: prompts, tools, caching per model

Harnesses ship as SDKs while framework vendors ship harnesses: the two categories are merging.

KEY CONCEPTS

- **Bottom up:** model, harness, platform, meta-harness.
- **Platform:** SDKs, durable libraries, ACP servers, skill marketplaces.
- **Meta-harness:** Omnigent ships adapters for 23 harnesses.
- **Implication:** extensions are a supply chain to secure.

The developer increasingly programs inside the harness.

Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §14 · Earendil (2026), Pi Durable, earendil.com/posts/pi-durable

TOPIC 5 / FROM SESSIONS TO ALWAYS-ON AGENTS

Recommendations from eleven harnesses

The study ends with 18 recommendations; seven connect directly to today's topics.

#	RECOMMENDATION	TOPIC	EVIDENCE TODAY
R1	Start with a linear loop	Harness	Pi: a ~790-line loop, no planner
R6	Auto-discover hierarchical Markdown context	Context	8 of 11 harnesses
R7	Compact at a threshold; keep a verbatim tail	Context	Hermes at 50%; Pi at window − 16k
R8	Do not build RAG over code	Patterns	0 of 11 use embedding search
R12	Stay single-agent until exploration needs parallelism	Patterns	Hermes: one child by default
R15	Do not use an agentic framework for the runtime	Harness	0 of 11 import one
R18	Ship cheap caps against stuck loops	Evaluation	Hermes: 90-step budget; Pi: none

KEY CONCEPTS

- **Recommendations** come from what eleven systems converged on.
- **Evidence column:** where today's systems follow or diverge.
- **R18 splits the field:** budgets versus trusting the model to stop.
- **Use them as defaults**, and know when you diverge.

Recommendations are evidence, not rules — know when you diverge.

Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §16

TOPIC 5 / FROM SESSIONS TO ALWAYS-ON AGENTS

A minimum viable harness in ~90 lines

The study closes with a scaffold that implements ten of its recommendations.

minimum_viable_harness.py

```python
# Condensed from the paper's ~90-line minimum viable harness
class Agent:
    async def run(self, task):
        self.messages = [system(discover_context()),  # AGENTS.md, root → leaf
                         user(task)]
        while True:
            self._check_limits()           # max turns, max cost
            await self._maybe_compact()    # summary + last 30% verbatim
            resp = await self.model.complete(self.messages, tools=SCHEMAS)
            self.messages.append(resp)
            if not resp.get("tool_calls"):
                return resp["content"]     # no tool call → done
            for call in resp["tool_calls"]:  # serial execution
                out = TOOLS[call["name"]](**call["args"])
                self.messages.append(tool_result(call, out))

TOOLS = {"bash": ..., "read_file": ...,
         "write_file": ..., "search_replace": ...}   # 4 tools
```

Condensed from Barbaste et al. (2026), Listing 3. Illustrative scaffold, not production code.

KEY CONCEPTS

- **The while loop** is the whole agent.
- **Limits and compaction** run before every model call.
- **Return when** the model stops calling tools.
- **Left out on purpose:** sandbox, multi-agent, MCP, skills.

Start minimal; add what your observed failures demand.

DISCUSSION

Which of today's five questions can our own project already answer?

HARNESS · PATTERNS · EVALUATION · CONTEXT · ALWAYS-ON

FURTHER READING

Koenigstein (2026)
Harness Engineering for AI Agents — the practitioner view

Grootendorst & Alammar
An Illustrated Guide to AI Agents

Earendil (2026)
Pi Durable — earendil.com/posts/pi-durable

Alejandro AO (2026)
Pi and Hermes architecture explained — alejandro-ao.com